MyCodeSchool

Code review — recapitulare-initiere-cpp

Vicențiu, aici sunt rundele de review pe proiectul tău, cea mai nouă sus.

Runda 6 — 2026-10-23 — sha 990b425

Ai reparat toată secțiunea 8 și ai scris 9 și 10 în aceeași zi. Am compilat fiecare header separat cu `clang++`, ca pe Mac-ul tău, și am rulat fiecare exercițiu pe exemplul din fișă, plus capetele, cu `-fsanitize=address,undefined`.

Poarta de compilare: ✅ `main.cpp` compilează (1 warning). Separat, toate headerele tale trec, mai puțin `cautare_binara.h` (3 erori, B17, neatins).

Fișa, secțiunile 6–10:

✅ corecte	6.1–6.5 · 7.1 · 7.2 · 8.1–8.5 · 9.2 · 9.3 · 10.1–10.5
🔴 rezultat greșit	9.1 (valoare mai mare decât toate) · 9.4 (valoare pară) · 9.5
⏳ nescrise	7.3 · 7.4 · 7.5

Închise din runda 5: B28 · B29 · B30 · B18 · B26 · M22, plus 8.4 (B31, în `stergere.h`). Detaliile sunt în tabelul rundei 5, mai jos.

Despre fișierele „care nu se vedeau”: `inserare.h` și `secventa_egale.h` erau scrise pe nume în `~/.gitignore_global`, fișierul de ignorare pentru **toate** repo-urile de pe Mac. Acolo pui doar gunoaie de sistem (`.DS_Store`). Ce nu vrei în git dintr-un proiect pui într-un `.gitignore` în proiect, vezi C18.

🔴 Critice

B32 🔴 9.5: scrii 12 valori într-un vector cu o singură căsuță `inserare.h:127`

CUM E ACUM: `int v[1];`, iar `interclasare` scrie în el `dx + dy` valori.

```
main.cpp (sol5)   ->  12 10 28 17 39 48 58 58 79 58 79
trebuia               12 10 28 17 39 48 58 59 79 80 86 89
ASan:  stack-buffer-overflow ... 'v' (line 127) <== overflows this variable
```

DE CE: `v[1]` rezervă o singură căsuță pe stivă. C++ nu verifică indicele: `v[5] = ...` scrie pur și simplu acolo unde ar fi fost căsuța 5, adică peste variabilele de lângă `v`. Asta înseamnă `x`, `y` și `d`. De aceea în ieșire apar `58` și `79` de două ori: au fost scrise peste valorile din `y`, iar apoi citite înapoi ca și cum ar fi fost din `y`. Programul nu se oprește cu eroare. Merge mai departe cu date stricate.

FIX: `v` trebuie să încapă `dx + dy` valori, deci `int v[200];`.

💡 Pe Mac poți vedea singur asta: `clang++ -g -fsanitize=address main.cpp`, apoi rulezi. Îți arată linia și variabila depășită.

B33 🔴 9.5: `if` la coadă copiază doar un element din ce rămâne `inserare.h:107` · `:113`

CUM E ACUM: după `while`, restul din `x` și din `y` se copiază cu `if`, deci se copiază un singur element.

```
a: 1 2 3   b: 7 8       ->  1 7 2 8 3     ✅ (rămâne 1)
a: 1       b: 7 8 9     ->  1 7 8         trebuia 1 7 8 9
datele tale, v[200]     ->  ... 79 80 86  lipsește 89
```

DE CE: `if` se execută o dată, `while` de câte ori e adevărată condiția. Când un vector e mai lung cu două sau mai multe elemente, rămân mai multe de copiat. Pe exemplul din fișă rămâne exact unul, deci `if` și `while` dau același rezultat și bug-ul nu se vede. În `interclasare.h` (7.1) ai scris coada cu `while`, și de aceea acolo merge.

FIX: `while (i < dx)` și `while (j < dy)`.

💡 Câte elemente din `y` au rămas necopiate pe datele tale după ce s-a terminat primul `while`?

B34 🔴 9.4: cu o valoare pară, inserarea nu se mai oprește `inserare.h:84`

CUM E ACUM: după ce inserezi pe `i + 1`, `for`-ul face `i++` și ajunge exact pe valoarea tocmai inserată.

```
5 8 3 2 , pun 99   ->  5 8 99 3 2 99                    ✅
5 8 3 2 , pun 100  ->  5 8 100 100 100 100 100 ...      (oprit de mine la d = 20)
```

DE CE: E oglinda lui B29. La ștergere, elementul de după alunecă pe `i` și îl săreai. La inserare, elementul nou stă pe `i + 1`, iar pasul următor îl verifică. Dacă e par, inserezi încă unul după el, apoi încă unul, iar `d` crește până trece de cele 100 de căsuțe. Cu `99` merge doar pentru că `99` e impar. La 9.2 ai rezolvat exact problema asta: `i += 2` sare peste copia pusă.

FIX: după `inserareElement(...)`, `i++` (sari peste elementul inserat).

💡 Ce face `i += 2` la 9.2 și de ce ai avut nevoie de el acolo?

B35 🔴 9.1: o valoare mai mare decât toate nu e inserată `inserare.h:36`

CUM E ACUM: inserezi doar când găsești un `v[i] > elem`.

```
2 5 9 14 , pun 7   ->  2 5 7 9 14    ✅
2 5 9 14 , pun 20  ->  2 5 9 14      trebuia 2 5 9 14 20
```

DE CE: Locul unei valori mai mari decât toate e după ultimul element, adică pe poziția `d`. Acolo nu există niciun `v[i]` mai mare, deci `if`-ul nu e adevărat niciodată. Bucla se termină fără inserare, iar `break` nu se execută. Programul nu dă eroare, valoarea doar lipsește.

FIX: poziția pornește de la `d` și o micșorezi când găsești un `v[i] > elem`. Apoi inserezi o singură dată, după buclă.

💡 Pe ce poziție trebuie pus `20` în `2 5 9 14`? Ajunge vreodată `for`-ul tău acolo?

B19 🔴 EXTINS: funcții `int` fără `return` `secventa_egale.h:73` · `:104`

CUM E ACUM: `ceaMaiLungaSecventaContor` și `ceaMaiLungaSecventaCrescatoare` sunt `int`, dar afișează cu `cout` și nu întorc nimic.

```
UBSan:  secventa_egale.h:73:5: runtime error: execution reached the end
        of a value-returning function without returning a value
```

DE CE: `int` în fața funcției e o promisiune: „întorc un număr". Dacă funcția ajunge la `}` fără `return`, în C++ e comportament nedefinit. Aici rezultatul iese bine doar pentru că `sol3` și `sol4` nu folosesc valoarea întoarsă. Fișa cere „întoarce", deci varianta corectă e cea de la 10.2: `return val;`, iar `cout` doar în `sol`.

FIX: 10.3: `return ct;`, iar `cout` în `sol3`. 10.4: `void` cu `start` și `lungime` prin `&`, ca la 10.1.

- același bug, neatins din runda 4: `cautare_binara.h:99`, `:116`, `:199`, plus `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h` (vezi B19 în tabelul rundei 4)

🟡 Importante

- **M24** — `secventa_egale.h:82`: `j - 1 < d` în loc de `j + 1 < d`. Pe un vector cu exact `d` căsuțe (`int a[3] = {1,2,3}`), ASan oprește programul cu `stack-buffer-overflow`: `v[j+1]` se citește

după capăt. Numărul iese bine doar pentru că vectorii tăi au 100 de căsuțe, cu zerouri după `d`. La 10.1, 10.2, 10.4 și 10.5 ai scris corect `j + 1 < d`.

- **M19** — la 8.x ai scos funcții cu parametri ✅. 9.1–9.4 sunt din nou scrise direct în `sol`.
- **M21** — `afisare` și `sol1` … `sol5` există și în `inserare.h`, și în `stergere.h`, și în `interclasare.h`. `#include "stergere.h"` + `#include "inserare.h"` → `error: redefinition of 'afisare'`.
- **M18** — 10.3, 10.4 (și 6.3, 6.5 din runda 5) afișează în loc să întoarcă.
- **M23** — 7.1 tot fără `sol`. 7.3, 7.4, 7.5 nescrise.
- **M1** — ✅ `sortare-interval.h`. `while-ex.h:1` și `parcurgerea-vectoriilor.h:1` au tot cratimă în gardă.

🟢 Cleanups

- **C18** — pe lângă `.exe` / `.o` ai comis acum și `main` și `main.dSYM/`. Un `.gitignore` în proiect cu `bin/`, `obj/`, `main`, `*.dSYM/` le oprește pe toate. Cele deja comise ies cu `git rm --cached`.
- **C28** — variabile nefolosite: `secventa_egale.h:11` (`int i, j;`), `:44`, `:75-76`, `inserare.h:77` (`int i;`).
- **C29** — 9.5 se numește `interclasare`, ca algoritmul din 7.1, dar nu interclasează, ci alternează: `alternare`.
- **C30** — `secventa_egale.h:1` are garda `SORTARE_EGALE_H_INCLUDED`; `inserare.h:41` `};` și `secventa_egale.h:81` `j = i;;`.
- **C27** — `stergere.h:146` tot „fectorul", „vectot".

Ce e bine

- **Secțiunea 8 reparată complet** și scoasă în funcții cu parametri (`stergereValoare`, `stergereRepetate`, `stergereSubMedie`): `5 2 5 7 2 2 9` -> `5 2 7 9`, `4 8 1 7` -> `8 7`, `10 14 10 15` -> `15`.
- **B28**: ai înțeles ordinea. Funcțiile de bază sunt acum sus, iar `munte_vale.h` și `interclasare.h` compilează singure.
- **Trasările**: interschimbarea `3` / `5` deasupra lui `sortareIntervalDescrescator`, pe care ai și reparat-o, și interclasarea pe datele tale, rând cu rând. Exact asta se cerea.
- **9.2 și 9.3**: `i += 2` ca să sari peste ce ai inserat.
- **Secțiunea 10**: din `ceaMaiLungaSecventaEgale` ai construit singur cele 4 variante, toate corecte pe exemplele din fișă.
- **Mesajele de commit** spun acum ce ai făcut („secvente si inserari").

Before / After (🔴)

	Before	After
B32	`int v[1];`	`int v[200];`
B33	`if(i < dx)` · `if(j < dy)`	`while(i < dx)` · `while(j < dy)`
B34	`inserareElement(v, d, i + 1, elem);`	`inserareElement(v, d, i + 1, elem); i++;`
B35	inserare doar când `v[i] > elem`	`poz = d`; prima `i` cu `v[i] > elem` → `poz = i`; apoi `inserareElement(v, d, poz, elem)`
B19	`int ...Contor(...) { ... cout << ct; }`	`int ...Contor(...) { ... return ct; }`, iar `cout` în `sol3`

Q&A — răspunde în scris, înainte de runda 7

1. La 9.4, cu valoarea `100`: ce verifică `for`-ul imediat după prima inserare și de ce nu se mai oprește? Ce ai făcut diferit la 9.2?

2. La 9.5: câte căsuțe are `int v[1]` și câte valori scrii în el? Unde ajung cele care nu încap?

3. La 9.1: pe ce poziție trebuie pus `20` în `2 5 9 14` și de ce `for`-ul tău nu ajunge niciodată să-l insereze?

(Răspunsurile din rundele 4 și 5 nu au apărut în repo.)

Runda 5 — 2026-10-23 — sha 129eeb4 (condensată)

Starea fiecărei constatări, verificată prin rulare pe `990b425`:

#	Loc	Constatare	Stare pe `990b425`
B28	`munte_vale.h`, `interclasare.h`	funcții chemate înainte de definiție, `MAX` nedefinit	✅ ÎNCHIS: ambele compilează separat

#	Loc	Constatare	Stare pe `990b425`
B29	`stergere.h:65`	`stergerePare` fără `i--`	✅ ÎNCHIS: `10 14 10 15` -> `15`
B30	`stergere.h:106`	8.3 sortează și citește `v[d]`	✅ ÎNCHIS: `5 2 5 7 2 2 9` -> `5 2 7 9`
B31	`stergere.h:126`, `sortare_bule.h:12`	medie cu împărțire întreagă	🔴 PE JUMĂTATE: 8.4 ✅ (`double s`, `double ma`); `mediana` pe `35 56 19 42` tot `38`
B17	`algoritmi.h:168`, 4 headere	`algoritmi.h` inclus și editat	🔴 NEATINS: 4 include-uri, `sortareMunte` din `algoritmi.h` tot `16 31 54 99 17 17`, `cautare_binara.h` 3 erori · ✅ partea din `sortare-interval.h:70` reparată, cu trasare
B27	`interclasare.h`, `munte_vale.h`, `sortare-interval.h`	funcții copiate din `algoritmi.h`	🟡 PE JUMĂTATE: interclasarea are acum trasarea ta ✅; `sortareInterval` și `sortareIntervalDescrescator` tot copii (doar `while` / `for` rescrise), iar `ceaMaiLungaSecventaEgale` (`secventa_egale.h:7`) e copiată, cu 2 variabile nefolosite în plus
B18	`sortare-interval.h:59`	descrescător -> `1 1 1 1 1 1`	✅ ÎNCHIS: `9 7 5 3 2 1`
B26	`sortare-interval.h:133`	`sortareTreiParti` de la 1	✅ ÎNCHIS: `9 7 5 3 1 2` -> `7 9 3 5 1 2`

#	Loc	Constatare	Stare pe `990b425`
B19–B25, B15	`cautare_binara.h`, `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h`, `parcurgerea-vectoriilor.h`	vezi runda 4	🔴 NEATINSE (fișierele identice cu `129eeb4`)
M18	`munte_vale.h:76`, `:140`	afișează în loc să întoarcă	🟡 NEATINS · extins la 10.3, 10.4
M19	`stergere.h`	8.x direct în `sol`	✅ pentru 8.x · revine la 9.x
M20	`stergere.h:152`	`f[v[i]]` cu `f[100]`	🟡 NEATINS
M21	—	nume duplicate între headere	🟡 NEATINS + `inserare.h`
M22	`interclasare.h:52-66`	sortare prin inserție, `MAX`, `const char*`	✅ ÎNCHIS: bule, `int x[100]`
M23	—	7.1 fără `sol`, 7.3–7.5 nescrise	🟡 NEATINS
M1	gărzile	cratimă	🟡 PE JUMĂTATE: `sortare-interval.h` ✅
C24	`stergere.h:131`	11 valori, `d = 10`	✅ ÎNCHIS
C25	`main.cpp`	cod mort comentat	✅ ÎNCHIS
C26	—	mesaje „.”	✅ ÎNCHIS
C18	`bin/`, `obj/`	binare în git	🟢 EXTINS: `main`, `main.dSYM/`
C27	`stergere.h:146`	typo-uri	🟢 NEATINS
Q&A	—	cele 3 întrebări ale rundei 5	niciun răspuns în repo

Runda 4 — 2026-10-15 — sha 6c66dca (condensată)

Poarta rundei 4: `main.cpp` compila; separat pica `cautare_binara.h` și, după editarea `algoritmi.h`, toate cele 4 headere dependente. Starea fiecărei constatări, verificată pe `129eeb4`:

#	Loc	Constatare	Stare pe `129eeb4`
B17	`algoritmi.h:168`, 4 headere	`algoritmi.h` inclus; interschimbare greșită	🔴 REDESCHIS: compilează prin editarea materialului, interschimbarea rămâne greșită (vezi runda 5)
B18	`sortare-interval.h:59`	`sortareIntervalDescrescator` pe `1 2 3 5 7 9` -> `1 1 1 1 1 1`	🔴 NEATINS
B19	`cautare_binara.h`, `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h`	funcții `int` fără `return`	🔴 NEATINS (fișierele identice cu `6c66dca`; `cautare_binara.h` tot necompilabil separat)
B20	`cautare_binara.h:142`, `:145`	`<` / `>` în loc de `<=` / `>=`	🔴 NEATINS (`cautare_binara.h` identic)
B21	`cautare_binara.h:48`, `:57`	căutare binară pe vector neordonat	🔴 NEATINS (`sol25`)
B22	`minim_maxim.h:64-69`	`aDouaValoare` compară valoare cu poziție	🔴 NEATINS: `9 3 8 1` -> `1` (trebuia 8); `3 3 8 1` -> 3, tot din întâmplare
B23	`sortare_bule.h:67`	`dejaOrdonat` sortează originalul	🔴 NEATINS: `dejaOrdonat(1 7 3)` -> 0, vectorul după apel `1 3 7`
B24	`sortare_selectie.h:26-35`	`ordonareDupaUc` afișează ultimele cifre	🔴 NEATINS: `23 41 15 32` -> `1 2 3 5`

#	Loc	Constatare	Stare pe `129eeb4`
B25	`sortare_selectie.h:49`	`v[i - 1]` la `i = 0`	🔴 NEATINS (identic)
B26	`sortare-interval.h:133`	`sortareTreiParti` pornește de la 1	🔴 NEATINS: `9 7 5 3 1 2` -> `9 7 3 5 1 2`
B27	`cautare_binara.h:108`, `sortare-interval.h:40`	funcții copiate din `algoritmi.h`	🔴 EXTINS: copiere și în `interclasare.h` și `munte_vale.h` (vezi runda 5)
B15	`parcurgerea-vectoriilor.h:183`	`produsCifre(234)` -> `4`	🔴 NEATINS
B16	`parcurgerea-vectoriilor.h:223`	„prime între ele” = amândouă prime	🔴 NEATINS (identic)
M1	`sortare-interval.h:1`, `while-ex.h:1`, `parcurgerea-vectoriilor.h:1`	garda cu cratimă	🟡 NEATINS
M14	—	`mediana`, `kValoare`, `primeleK` sortează vectorul primit	🟡 NEATINS
M4	`while-ex.h:345`	`cmmmc(50000, 70000)` -> `-79496`	🟡 NEATINS
M17	—	`cifraMaxima(0)` -> `-1`	🟡 NEATINS
M16	`interclasare.h`, `munte_vale.h`, `stergere.h`, `sortate_bule.h`, `fisa2.h`	fișiere goale comise	🟡 ✅ primele 3 au cod · `sortate_bule.h` și `fisa2.h` tot goale
C18	`bin/`, `obj/`	binare în git	🟢 NEATINS, recomise în `d47270f`

#	Loc	Constatare	Stare pe `129eeb4`
C20–C23	—	`afisareDistinceDescrescatoare`, variabile nefolosite, `primeleK` cu `k > dim`, mesaje de commit	🟢 nerevizuite în runda 5 (C23 continuă ca C26)
Restanțe r1	`while-ex.h`	B3, B4, B6, M2, M3, M6	nerevizuite în runda 5
Q&A	—	cele 3 întrebări ale rundei 4	niciun răspuns în repo

Runda 3 — 2026-10-01 — sha ee7010e (condensată, nelivrată la timp)

Generată pe 2026-10-01, n-a ajuns la tine. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B15	`parcurgerea:183`	`produsCifre`: `suma * n % 10` → mereu o cifră	🔴 NEATINS (vezi runda 4)
B16	`parcurgerea:223`	„prime între ele" verificat cu `isPrim`	🔴 NEATINS (vezi runda 4)
B9	`parcurgerea:55-57`	`isPrim(i)` în loc de `isPrim(v[i])`	✅ ÎNCHIS
M10	`parcurgerea:109`	`afisareRasturnat` testează doar capetele	✅ ai ales palindromul: `punctul_c` cu `ePalindrom`
M11	`parcurgerea:156`	`sol1d` nu poate afișa ramura adevărată	🟡 NEATINS
M12	`parcurgerea:20`, `:253`	Cod comentat care nu compilează	🟡 `punctul_c` rescris ✅; `sol1` și `punctul_g` tot comentate

#	Loc	Constatare	Stare pe `6c66dca`
M7	—	`isPrim` ≡ `estePrim`, `CifraMinima` ≡ `cifraMinima`	🟡 NEATINS
C14	`parcurgerea:107`, `:207`	Variabile nefolosite	🟢 NEATINS
C15	`parcurgerea:168`	`?:` cu `<<`	🟢 NEATINS
C16	`parcurgerea:142`	`divizoriProprii` întoarce un număr	🟢 NEATINS
C17	—	`soll2`	✅ redenumit `sol2`
C18	`bin/`, `obj/`	Binare în git	🟢 NEATINS

Runda 2 — 2026-10-01 — sha fdda59a (condensată)

Textul integral e în PDF-ul rundei 2 (commit `b3b1bf6`).

#	Constatare	Stare pe `6c66dca`
B8	Nu compila, 7 erori	✅ ÎNCHIS (runda 3)
B9	`contorPrimeVector` număra indicii	✅ ÎNCHIS
B10	Indexare de la 1 vs de la 0	✅ ÎNCHIS
B11	Palindromul testa doar capetele	✅ ÎNCHIS
B12	`punctul_g` — buclă infinită	✅ ÎNCHIS
B13	`if` în afara buclei	✅ ÎNCHIS
B14	Funcție `int` fără `return`	✅ `afisareaElemVect` e `void` · 🔴 tiparul revine în runda 4 (B19)

#	Constatare	Stare pe `6c66dca`
M7	Funcții rescrise, existente deja	🟡 NEATINS · copiere în runda 4 (B27)
M8	`int c[10]` cu indexare de la 1	✅ codul a fost rescris
M9	`punctul_g` folosește `nr` nesetat	🟡 cod comentat
C7	`CIfraMinima`	✅ `CifraMinima`
C8	`//todo:` învechit	✅
C9	`dim` declarat, literal pasat	🟢 NEATINS (`parcurgerea:69`)
C10, C11	`main.cpp` vechi	✅ `main.cpp` rescris
C12	Binare în git	🟢 NEATINS
C13	Mesaje de commit	🟢 NEATINS

Runda 1 — 2026-09-24 — sha 2734bb4 (condensată)

Livrată pe 2026-10-01, odată cu runda 2. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B1	`main.cpp:1`	`while-ex.h` nu ajungea la compilator	✅ compilat separat de mine: 0 erori
B2	`while-ex.h:72`, `:429`, `:352`	Cifra scoasă înainte de `while`	✅ ÎNCHIS în toate cele 3 locuri
B3	`while-ex.h:126`, `:171`	Două bucle cu `n` nemodificat → nu se termină	🔴 NEATINS
B4	`while-ex.h:124`, `:169`	Funcții `int` fără `return`	🔴 2 rămase · vezi B19

#	Loc	Constatare	Stare pe `6c66dca`
B5	`while-ex.h:303`	`cmmdc(0, 0)` → împărțire la zero	✅ ÎNCHIS
B6	`while-ex.h:452`	`eliminareCifra` scoate toate aparițiile	🔴 NEATINS
B7	`while-ex.h:370`	`celMaiMicNrCuSumaCifrelor(20)` → 99	✅ ÎNCHIS (funcția nouă)
M1	gărzile `.h:1`	Cratimă în numele macro-ului	🔴 NEATINS + `sortare-interval.h`
M2	`while-ex.h:154`, `:201`	`% p` în loc de `% 10`	🔴 NEATINS
M3	`while-ex.h:479`, `:554`	`pow` fără `<cmath>`	🔴 NEATINS
M4	`while-ex.h:345`	`a * b` → depășire	🔴 reprodus: `cmmmc(50000,70000)` → `-79496`
M5	`fisa2.h`	Fișier gol	🔴 NEATINS · vezi M16
M6	toate `.h`	Definiții în `.h`	🟡 NEATINS
C1–C3	—	`dim`/literal, `int n` nefolosit, `isZero`	🟢 NEATINSE
C4	`bin/`, `obj/`	Binare în git	🟢 NEATINS
C5	`while-ex.h:8`, `main.cpp:3`	`//todo:` învechite	🟢 `main.cpp` ✅, `while-ex.h:8` rămas
C6	—	Mesaje de commit	🟢 NEATINS